

CSC490: Assignment 5

Team Members

Name	Student ID
Aarya Prakash	1008790257
Derek Huynh	1008849052
Merrick Liu	1008951920
Rhys Balevicius	1000801974

University of Toronto
Department of Computer Science

1 Context for Our Project: Lighthouse

Lighthouse is a code search engine exposed via the Model Context Protocol (MCP) that improves the performance of AI coding agents by surfacing relevant context from a codebase. A developer selects GitHub repositories to index, Lighthouse ingests and indexes them, and the developer adds the Lighthouse MCP server to their AI coding agent. When the agent is working on a task and needs context that falls outside its immediate working scope, it queries Lighthouse to retrieve relevant code snippets, interfaces, and documentation.

The system is composed of several services: an **ingestion service** that clones repositories, chunks source files, generates embeddings, and stores them in Postgres and Milvus (vector database) via Temporal (distributed, durable execution solution) workflows; a **search service** that implements retrieval strategies (e.g. hybrid search combining dense vector similarity with sparse keyword matching); an **MCP server** that exposes search as MCP tools consumable by coding agents; and a **web service** for management and visualization (currently used for debugging).

1.1 Current State

We have implemented a baseline version of the full pipeline: repositories can be indexed (with both full and incremental indexing), searched via the MCP server, and results are returned to coding agents. However, several areas are still in active development:

- **Web UI** — The web service currently serves as a debugging interface for internal development. We are still discussing and formulating what the production UI will look like, and expect to redesign it in the coming week.
- **Evaluation suite** — We are developing an eval suite to measure how effectively Lighthouse improves coding agent outcomes, building on the aspirational dataset design from A2. This will let us quantitatively compare retrieval strategies.
- **Search and ingestion experimentation** — We are experimenting with different search strategies, chunking approaches (e.g. sliding window vs. AST-aware chunking), and embedding models to improve retrieval quality beyond our current baseline.

1.2 Tech & Testing Stack

Below are quick descriptions of each services (found in `services/`) tech stack:

- **ingestion** — A Python 3.11 service built with FastAPI and Temporal. It uses GitPython for repository cloning, our shared workspace packages (`shared`, `db`, `vectordb`, and `embedding`) for schemas and storage, and implements the indexing pipeline including chunking, embedding generation, and workflow orchestration.
- **search** — A Python 3.11 FastAPI service that implements retrieval logic. It uses the same shared workspace packages as the ingestion service and contains our search strategies, including hybrid search over Postgres and Milvus.

- **mcp_server** — A Python 3.11 service built with FastAPI and FastMCP. It exposes Lighthouse functionality as MCP and HTTP endpoints for coding agents, and also uses `httpx`, `cryptography`, `asyncpg`, and `boto3` for integration with authentication, persistence, and external services.
- **web** — A TypeScript frontend built with React 19 and Vite. It uses React Router for navigation, SWR for client-side data fetching, and Tailwind CSS for styling; at the moment it primarily serves as an internal debugging and management UI.

For testing, our testing framework is `pytest`, with tests organized into `unit`, `integration`, and `e2e` markers. We use `pytest-asyncio` for async service and API tests, and `pytest-cov` to measure line coverage across the Python services and shared packages. For infrastructure-dependent tests, we use `testcontainers` to provision isolated Postgres and Milvus instances during test runs instead of depending on shared developer machines. Since Temporal is a core part of the ingestion pipeline, we also use Temporal Python SDK testing utilities such as `temporalio.testing.WorkflowEnvironment` and `ActivityEnvironment` to test workflow orchestration and activities in a deterministic local environment.

2 Profiling Execution time

We performed profiling benchmarks of some significant hot paths in our function, focusing mostly on our intensive ingestion and search services. The raw benchmark results of the pre-improvement baselines can be seen in Appendix A and the improved performance data can be seen in Appendix B.

1. SlidingWindowChunker.chunk_file

In the baseline profile (`profiling_results.tmp`), `chunk_file` accounted for a cumulative 0.010s across 3 calls. The function itself was not dominated by loop control logic; instead, the profile showed repeated per-chunk work from hashing and string processing. In particular, `compute_chunk_hash` contributed 0.003s cumulative, with `hashlib.openssl_sha256` at 0.002s. This indicated that for medium-to-large chunk counts, hash choice materially affected end-to-end chunking latency.

The optimization focused on replacing SHA-256 with xxHash64 in `compute_chunk_hash`, while keeping the chunking algorithm and output semantics unchanged. This is appropriate because the hash is used for chunk identity/dedup behavior rather than cryptographic verification, so we don't need the full security guarantees of something like SHA-256. xxHash64 significantly reduces CPU cost per hash invocation while still providing practical collision resistance for this operational context. xxHash also allows for a larger 128-bit option which would reduce the chance of hash collisions significantly, though only extremely large codebases would even run the risk of collision in a 64-bit space.

In the post-change profile (`profiling_results_after_improvements.tmp`), `chunk_file` dropped to 0.006s cumulative across 3 calls (about 40% faster). Hashing is no longer a visible dominant cost: `compute_chunk_hash` and `xxhash.xxh64_hexdigest` both register effectively as near-zero at this scale. The remaining visible work is mostly `splitlines`, `join`, and baseline iteration overhead.

2. GitOperations.list_files

Before optimization, `list_files` took 0.216s cumulative across 2 calls. The profile showed expensive traversal behavior centered on `pathlib.rglob` and related selector machinery: `rglob` at 0.081s, selector internals at 0.078s, and heavy `stat/is_file` traffic (`stat` at 0.077s, `is_file` at 0.059s). The key issue was traversal and path-object overhead rather than filtering logic alone.

The implementation was changed to use `os.walk` traversal with inline filtering. This avoids the `pathlib` selector/`rglob` overhead, reduces object churn, and keeps filtering close to candidate discovery. The behavior contract was retained: same extension checks, same skip patterns, same size threshold, and exclusion of `.git`-internal content.

After optimization, `list_files` improved to 0.158s cumulative across 2 calls (about 27% faster). The hotspot shape also shifted as expected: `rglob` and selector frames disappeared, replaced by `os.walk` and direct path/stat operations. This confirms that the improvement came from cheaper traversal mechanics, not from reduced output scope

(`listed_files` remained 4860 in both runs).

3. `ChunkService.move_to_final`

In the baseline, `move_to_final` consumed 1.822s cumulative for the 8k-chunk profile case. Most of that time sat in ORM SQL generation and execution pathways: SQL construction (1.036s), internal SQL building (1.035s), and insert generation (1.027s). Additional cost came from staging row materialization and conversion (`process_row`, `python_value`, date/time adaptation paths). This profile clearly identified the expensive portions as ORM shaping plus per-row conversion overhead.

The maintainable optimization path preserved Peewee inserts (raw SQL insert path was intentionally reverted) and instead reduced per-row overhead where safe:

- read only required staging columns,
- fetch staging rows as tuples instead of full model instances,
- centralize embedding normalization/decoding in a lightweight helper.

These changes keep transactional behavior and output semantics intact while lowering model-instantiation and field-conversion pressure.

After these adjustments, `move_to_final` dropped to 1.260s cumulative (about 31% faster). ORM SQL generation remains the dominant cost center, but the total envelope is significantly reduced. The profile confirms reduced pressure from row-conversion-heavy portions compared to baseline, while preserving current maintainability constraints.

4. `HybridSearchStrategy.search`

Baseline search profiling showed `search` at 0.062s cumulative for 2 calls, with the main internal hotspot concentrated in publish filtering:

`_filter_vector_results` at 0.042s and `_get_active_publish_map` at 0.042s. This pointed to query-shape inefficiency (large OR predicate construction/execution) rather than fusion or snippet rendering. Supporting ORM SQL frames reinforced that diagnosis. The optimization replaced OR-heavy publish-map lookup with a grouped and batched strategy:

- deduplicate (*repository*, *branch*, *file_path*) keys before querying,
- group lookups by (*repository*, *branch*),
- issue batched `IN (...)` queries for file paths.

Additional low-risk trims were included:

- cap vector overfetch to prevent unnecessary widening in lookup workloads,
- build score map only for top-*k* fused entries used for final snippet scoring.

After optimization, overall `search` time dropped to `0.023s` cumulative for 2 calls (about 63% improvement). Internal hotspot reductions are substantial: `_get_active_publish_map` decreased from `~0.042s` to `~0.005s`, and `_filter_vector_results` decreased from `~0.042s` to `~0.006s`. Fusion remained inexpensive, confirming that the dominant gain came from improved DB query shape and key-set handling.

5. `ChunkService.publish_incremental_batch`

Baseline profiling showed `publish_incremental_batch` at `2.530s` cumulative for 3 calls in the 8k-chunk / 200-file workload. The dominant costs were concentrated in ORM-heavy paths and per-file upsert logic: SQL generation (`1.518s`), insert generation (`0.954s`), and repeated per-file `get_or_create` activity (`0.580s` cumulative for 600 calls across repeats). Additional overhead came from row materialization/conversion (`process_row`, `python_value`, date parsing/formatting paths), indicating that object hydration and query construction were significant contributors.

The implementation was optimized without changing external behavior. First, staging reads were moved to a column-limited `select(...).tuples()` shape so the method operates on lightweight tuple rows instead of hydrated model instances. Second, file tracking was reduced from a per-file list map to a set of staged file paths, since only membership checks are required for publish decisions. Third, embedding decoding was normalized through the shared `_decode_embedding` path to remove duplicate branch/convert logic in this function. Fourth, chunk inserts remained batched via Peewee but are now built from tuple rows, reducing repeated attribute access overhead. Fifth, the per-file `get_or_create + save` loop was replaced with batched `insert_many(...).on_conflict(update=...)` upserts on `indexed_files`, updating `active_publish_id` and `updated_at` in bulk.

After these changes, the same benchmark improved from `2.530s` to `1.447s` cumulative, a reduction of about **43%**. The hotspot profile still shows ORM SQL generation as the leading cost center, but the expensive per-file upsert pattern and model-materialization overhead were substantially reduced while preserving publish semantics and cleanup behavior.

3 Showing Code Coverage

We use `pytest-cov` to measure line coverage across all Python source packages. Coverage is automated via a GitHub Actions workflow at [.github/workflows/tests.yml](#), which runs on every push to `main` and on every pull request.

The workflow performs the following steps:

1. Runs `uv run pytest --cov --cov-report=xml --cov-report=term-missing` to execute all 293 tests and produce a coverage report.
2. Parses the generated `coverage.xml` to extract the overall line coverage percentage and generate a colored badge.
3. **On pushes to main:** updates the [README.md](#) coverage badge in-place (between `coverage:start` / `coverage:end` markers) and commits the change automatically.
4. **On pull requests:** posts (or updates) a comment on the PR with the current coverage percentage, so reviewers can see coverage impact at a glance.

4 Test Coverage Breadth

Our test suite consists of 300+ tests (unit, integration, and end-to-end) achieving **100% line coverage** across 2,418 source lines. As we push changes to the codebase, this may not be reflected in the `main` branch while grading, but you may view the coverage comment in our [A5 PR](#) to see that we achieved full coverage on submission.

4.1 What We Measure

Coverage is configured in [pyproject.toml](#) under `[tool.coverage.run].source` and includes all Python backend code:

- `packages/db/src` — Peewee ORM models and database utilities
- `packages/vectordb/src` — Milvus vector database client wrapper
- `packages/embedding/src` — Embedding provider abstraction (OpenAI, mock)
- `packages/shared/src` — Shared data models and utilities
- `services/ingestion/src` — Temporal workflows, activities, and chunking pipelines
- `services/search/src` — Search strategies (hybrid search)
- `services/mcp_server/src` — MCP server, HTTP/MCP routers, auth, and decorators

4.2 What We Exclude and Why

`services/web/` is excluded from coverage measurement. This service is a TypeScript/React frontend that currently serves as a debugging UI for internal development. It is not part of our Python test suite and is being actively redesigned in the coming week, so investing in test coverage for code that will be replaced would not be productive. Once the new frontend stabilizes, we plan to add Playwright or similar end-to-end tests for it separately.

We also use standard `exclude_lines` patterns in our coverage configuration to skip lines that are not meaningfully testable: `TYPE_CHECKING` blocks, `raise NotImplementedError` stubs, ellipsis-only bodies, and `if __name__ == "__main__"` guards.

5 Test Coverage Depth

Our most complex module is the `IndexBranchWorkflow` in the ingestion service ([services/ingestion/src/ingestion/temporal/workflows/index_branch.py](https://github.com/elastic/elasticsearch/blob/main/services/ingestion/src/ingestion/temporal/workflows/index_branch.py)). This Temporal workflow orchestrates 7 distinct activities through a multi-step pipeline: marking a branch as indexing, cloning the repository, chunking files into a staging table, deleting old chunks, embedding chunks in parallel batches, storing results to the final tables and vector database, and updating the branch status. It includes conditional early-exit logic (when no chunks are produced), parallel fan-out for embedding batches (using `asyncio.gather`), and a two-phase error handler that marks the branch as failed and conditionally cleans up staging data only when a `batch_id` has been assigned.

All 12 tests are located in [services/ingestion/tests/unit/test_index_branch_workflow.py](https://github.com/elastic/elasticsearch/blob/main/services/ingestion/tests/unit/test_index_branch_workflow.py) and use Temporal's `WorkflowEnvironment` with mocked activities. This isolates the workflow's orchestration logic from external dependencies (Postgres, Milvus, git, OpenAI), allowing us to test decision-making, parameter threading, and failure handling. Each test and the reasoning behind why we chose it is commented in the code.

5.1 Boundary Tests (Positive)

These tests target the embed batching loop, which uses `range(0, chunk_count, EMBED_BATCH_SIZE)` where `EMBED_BATCH_SIZE = 512`. Off-by-one errors in this loop would cause chunks to be silently skipped or duplicated.

1. **test_single_chunk_produces_one_embed_batch** - `chunk_count=1` produces exactly 1 embed call at offset 0. Tests the lower boundary of the batching loop.
2. **test_exactly_512_chunks_one_batch** - `chunk_count=512` (exactly `EMBED_BATCH_SIZE`) produces exactly 1 batch. This is the upper boundary for a single batch; a fencepost error would produce 0 or 2 batches.
3. **test_513_chunks_two_batches** — `chunk_count=513` is the smallest value that requires 2 batches. Verifies offsets are `[0, 512]`.
4. **test_large_chunk_count_batch_offsets** - `chunk_count=2049` produces 5 batches with offsets `[0, 512, 1024, 1536, 2048]`. Validates batch arithmetic at scale.

5.2 Failure Mode Tests (Negative)

These tests verify that the workflow correctly handles exceptions at different stages. The key invariant: `cleanup_staging` is called if and only if `batch_id` was set (i.e., `chunk_files` succeeded).

5. **test_delete_failure_triggers_cleanup** - `delete_existing_chunks` fails *after* `batch_id` is assigned. Verifies both `status="failed"` and `cleanup_staging` are called. This failure point was not covered by existing e2e tests.
6. **test_store_failure_triggers_cleanup** - `store_chunks` fails after embedding succeeds. Embedded staging rows must be cleaned up to avoid dangling data.

7. **test_chunk_files_failure_no_cleanup** - `chunk_files` raises *before* `batch_id` is assigned. Verifies `cleanup_staging` is **not** called (there is nothing to clean), validating the `if batch_id is not None` guard in the exception handler.

5.3 Parameter Propagation Tests (Positive)

These tests verify that input parameters are correctly threaded through to the right activities. Silent parameter drops would cause production failures with no error signal.

8. **test_github_token_propagated** - A `github_token` must reach both `update_branch_status` (for private repo branch creation via GitHub API) and `git_clone_or_fetch` (for authenticated clone). Dropping the token causes silent 401 errors on private repos.
9. **test_custom_strategies_propagated** - Custom `chunker_strategy="ast"` and `embedding_strategy="cohere"` must be forwarded to `chunk_files` and `embed_chunk_batch` respectively. If silently replaced by defaults, the user gets wrong results with no error.
10. **test_latest_commit_in_final_status** - The final `update_branch_status(status="indexed")` must include `latest_commit` from the git output, while the initial `status="indexing"` call must have `latest_commit=None`. Swapping these would record a stale or null commit SHA.

5.4 Cleanup Correctness and Return Value Tests

11. **test_cleanup_receives_correct_batch_id** - On failure after chunking, `cleanup_staging` must receive the exact `batch_id="batch-001"` returned by `chunk_files`. A wrong batch ID would leak staging rows while potentially purging unrelated data.
12. **test_return_value_format** - With `full_name="myorg/myrepo"` and `branch="feature/cool"`, the return string must be `"Indexed myorg/myrepo/feature/cool at abc123"`. Branch names with slashes must not be truncated. This serves as a regression test for string interpolation used in observability logging.

6 Two Memorable Bugs

6.1 Links

- Here is a [video walk through](#) of the two memorable bugs.
- Here is a [link](#) to the slides shown in the video.

6.2 Summary

Context: The [IncrementalIndexWorkflow](#) is a Temporal workflow triggered by GitHub webhooks that re-indexes changed repository files into the search pipeline. On each invocation it clones the updated branch, chunks modified files, generates embeddings, and writes results into Postgres and Milvus so that search reflects the latest code.

Bug 1: Temporary Gap in Searchable Data. The workflow deleted existing search chunks for changed files before their replacements were ready, creating a window during which a recently modified file was invisible to search. The fix introduced a publish-and-switch model: new chunks are staged under a `publish_id` and only promoted to active once fully written, so old data remains searchable until the new version is ready. This taught us that correctness is not only about eventually producing the right data. It is also about what is visible at every intermediate state. It also showed us that background tasks can also have big ripple effects.

Bug 2: Catastrophic Deletion on Failure. If any step failed after deletion (during chunking, embedding, or storage) the workflow would terminate without restoring the removed chunks, leaving those files absent from search until a future successful re-index. The publish-and-switch architecture resolves this as well: because deletion no longer precedes the publish, a failure at any point leaves the active search data exactly as it was before the workflow began. This highlighted that orchestration order is crucial and retry logic introduces complexity.

A Profiling Benchmarks Baseline

```

=> SlidingWindowChunker
    3758 function calls in 0.010 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall  filename:lineno(function)
      3    0.001    0.000    0.010    0.003  sliding_window_chunker.py:20(chunk_file)
    468    0.000    0.000    0.003    0.000  utils.py:6(compute_chunk_hash)
      3    0.003    0.001    0.003    0.001  {method 'splitlines' of 'str' objects}
    468    0.002    0.000    0.002    0.000  {built-in method _hashlib.openssl_sha256}
      3    0.001    0.000    0.001    0.000  {method 'strip' of 'str' objects}
    468    0.001    0.000    0.001    0.000  {method 'encode' of 'str' objects}
      1    0.000    0.000    0.000    0.000  {method 'join' of 'str' objects}
    468    0.000    0.000    0.000    0.000  {method 'hexdigest' of '_hashlib.HASH' objects}
    468    0.000    0.000    0.000    0.000  <string>:2(__init__)
    468    0.000    0.000    0.000    0.000  {built-in method builtins.min}
    468    0.000    0.000    0.000    0.000  {method 'append' of 'list' objects}
      3    0.000    0.000    0.000    0.000  {built-in method builtins.len}
      1    0.000    0.000    0.000    0.000  {built-in method builtins.max}
      1    0.000    0.000    0.000    0.000  {method 'disable' of '_lsprof.Profiler' objects}

generated_chunks=156 repeat=3 line_count=25000

=> GitOperations.list_files
    382556 function calls (382305 primitive calls) in 0.216 seconds

Ordered by: cumulative time
List reduced from 90 to 40 due to restriction <40>

ncalls  tottime  percall  cumtime  percall  filename:lineno(function)
      2    0.021    0.010    0.216    0.108  git_ops.py:70(list_files)
   19802    0.005    0.000    0.083    0.000  pathlib.py:1008(stat)
   10082    0.002    0.000    0.081    0.000  pathlib.py:956(rglob)
   10082    0.008    0.000    0.078    0.000  pathlib.py:402(_select_from)
   19802    0.054    0.000    0.077    0.000  {built-in method posix.stat}
   10080    0.006    0.000    0.059    0.000  pathlib.py:1261(is_file)
   10202    0.015    0.000    0.031    0.000  pathlib.py:353(_select_from)
   20160    0.013    0.000    0.028    0.000  pathlib.py:583(__hash__)
   20046    0.006    0.000    0.024    0.000  pathlib.py:546(__fspath__)
   20046    0.011    0.000    0.018    0.000  pathlib.py:536(__str__)
   10080    0.013    0.000    0.014    0.000  pathlib.py:569(_cparts)
   9840    0.007    0.000    0.013    0.000  pathlib.py:630(suffix)
   10200    0.005    0.000    0.010    0.000  pathlib.py:877(
_make_child_relp
364/124    0.006    0.000    0.008    0.000  pathlib.py:383(
_iterate_directories)
   20040    0.007    0.000    0.008    0.000  pathlib.py:622(name)
   10200    0.006    0.000    0.007    0.000  pathlib.py:523(
_format_parsed_parts)
   9960    0.007    0.000    0.007    0.000  pathlib.py:745(parts)
   10200    0.004    0.000    0.005    0.000  pathlib.py:515(_from_parsed_parts)
      1    0.000    0.000    0.000    0.000  {method 'add' of 'set' objects}
   10080    0.003    0.000    0.004    0.000  {method 'fullmatch' of 're.Pattern' objects}
29781/29778    0.002    0.000    0.002    0.000  {built-in method builtins.len}
      244    0.000    0.000    0.002    0.000  pathlib.py:934(_scandir)
      244    0.001    0.000    0.002    0.000  {built-in method posix.scandir}
   20160    0.002    0.000    0.002    0.000  pathlib.py:327(_select_from)
   10201    0.002    0.000    0.002    0.000  {method 'join' of 'str' objects}
   10080    0.001    0.000    0.001    0.000  {built-in method builtins.hash}
   9840    0.001    0.000    0.001    0.000  {method 'rfind' of 'str' objects}

```

```

10200      0.001      0.000      0.001      0.000 {built-in method __new__ of type
object at 0x561701efbf10}
10080      0.001      0.000      0.001      0.000 pathlib.py:257(casefold_parts)
10080      0.001      0.000      0.001      0.000 {method 'lower' of 'str' objects}
10080      0.001      0.000      0.001      0.000 {built-in method _stat.S_ISREG}
9738       0.001      0.000      0.001      0.000 {method 'append' of 'list'
objects}
2          0.001      0.000      0.001      0.000 {method 'clear' of 'set' objects}
10080      0.001      0.000      0.001      0.000 {method 'is_dir' of 'posix.
DirEntry' objects}
2/1        0.000      0.000      0.000      0.000 pathlib.py:281(_make_selector)
1          0.000      0.000      0.000      0.000 pathlib.py:380(__init__)
2/1        0.000      0.000      0.000      0.000 pathlib.py:304(__init__)
1          0.000      0.000      0.000      0.000 pathlib.py:349(__init__)
1          0.000      0.000      0.000      0.000 pathlib.py:260(compile_pattern)
1          0.000      0.000      0.000      0.000 __init__.py:225(compile)

listed_files=4860 repeat=2 dirs=60 files_per_dir=80

==> ChunkService.move_to_final
4603292 function calls (4417205 primitive calls) in 1.822 seconds

Ordered by: cumulative time
List reduced from 310 to 40 due to restriction <40>

ncalls      tottime  percall  cumtime  percall  filename:lineno(function)
1           0.021    0.021    1.821    1.821    chunk.py:101(move_to_final)
18          0.000    0.000    1.158    0.064    peewee.py:2004(inner)
18          0.000    0.000    1.158    0.064    peewee.py:2079(execute)
17          0.002    0.000    1.157    0.068    peewee.py:2595(_execute)
18          0.000    0.000    1.156    0.064    peewee.py:3309(execute)
16          0.000    0.000    1.128    0.070    peewee.py:2881(_execute)
104768/18    0.042    0.000    1.036    0.058    peewee.py:632(sql)
16          0.007    0.000    1.035    0.065    peewee.py:2848(__sql__)
16          0.285    0.018    1.027    0.064    peewee.py:2723(_generate_insert)
8035/35      0.060    0.000    0.592    0.017    peewee.py:1868(__sql__)
97417/24333  0.007    0.000    0.512    0.000 {built-in method builtins.len}
1           0.000    0.000    0.510    0.510    peewee.py:2110(__len__)
1           0.000    0.000    0.510    0.510    peewee.py:4706(__len__)
1           0.003    0.003    0.510    0.510    peewee.py:4739(fill_cache)
16002       0.008    0.000    0.509    0.000    peewee.py:4799(next)
8001        0.009    0.000    0.500    0.000    peewee.py:4713(iterate)
8000        0.018    0.000    0.467    0.000    peewee.py:8054(process_row)
96002       0.036    0.000    0.412    0.000    peewee.py:1444(__sql__)
96002       0.186    0.000    0.375    0.000    peewee.py:644(value)
8000        0.038    0.000    0.366    0.000    peewee.py:8012(process_row)
104000      0.024    0.000    0.329    0.000    peewee.py:4966(python_value)
24000       0.008    0.000    0.290    0.000    peewee.py:5445(adapt)
8000        0.016    0.000    0.280    0.000    peewee.py:5410(format_date_time)
24000       0.009    0.000    0.263    0.000 {built-in method strftime}
24000       0.020    0.000    0.254    0.000 _strptime.py:564(
_strptime_datetime)
24000       0.134    0.000    0.233    0.000 _strptime.py:309(_strptime)
18          0.000    0.000    0.118    0.007    peewee.py:3302(execute_sql)
18          0.095    0.005    0.118    0.007 {method 'execute' of 'sqlite3.
Cursor' objects}
224384      0.067    0.000    0.085    0.000 peewee.py:4939(__hash__)
8000        0.021    0.000    0.081    0.000 peewee.py:6778(__init__)
8000        0.005    0.000    0.078    0.000 __init__.py:299(loads)
8000        0.012    0.000    0.071    0.000 decoder.py:332(decode)
104002      0.020    0.000    0.059    0.000 {built-in method builtins.setattr}
24000       0.008    0.000    0.056    0.000 _strptime.py:26(_getlang)
642147      0.046    0.000    0.055    0.000 {built-in method builtins.
isinstance}
192291      0.042    0.000    0.054    0.000 peewee.py:562(__getattr__)
208542      0.043    0.000    0.053    0.000 peewee.py:640(literal)
8000        0.053    0.000    0.053    0.000 decoder.py:343(raw_decode)
24000       0.014    0.000    0.048    0.000 locale.py:594(getlocale)
96002       0.024    0.000    0.047    0.000 peewee.py:4963(db_value)

moved=8000 final_count=8000 milvus_inserted=8000 target=move_to_final

```

```

==> ChunkService.publish_incremental_batch

6567890 function calls (6279793 primitive calls) in 2.530 seconds

Ordered by: cumulative time
List reduced from 363 to 30 due to restriction <30>

ncalls  tottime  percall  cumtime  percall  filename:lineno(function)
      3      0.027      0.009      2.530      0.843  chunk.py:199(
publish_incremental_batch)
      1425      0.001      0.000      1.608      0.001  peewee.py:2004(inner)
      1425      0.001      0.000      1.606      0.001  peewee.py:2079(execute)
      1425      0.003      0.000      1.591      0.001  peewee.py:3309(execute)
198973/1625      0.094      0.000      1.518      0.001  peewee.py:632(sql)
      819      0.004      0.000      1.272      0.002  peewee.py:2595(_execute)
      216      0.000      0.000      1.062      0.005  peewee.py:2881(_execute)
      216      0.007      0.000      0.967      0.004  peewee.py:2848(__sql__)
      216      0.257      0.001      0.954      0.004  peewee.py:2723(_generate_insert)
14453/2653      0.076      0.000      0.831      0.000  peewee.py:1868(__sql__)
      17409      0.009      0.000      0.593      0.000  peewee.py:4799(next)
      9406      0.011      0.000      0.583      0.000  peewee.py:4713(iterate)
      600      0.005      0.000      0.580      0.001  peewee.py:6990(get_or_create)
      603      0.005      0.000      0.561      0.001  peewee.py:4739(fill_cache)
      8800      0.021      0.000      0.538      0.000  peewee.py:8054(process_row)
146606/73522      0.010      0.000      0.508      0.000  {built-in method builtins.len}
      3      0.000      0.000      0.503      0.168  peewee.py:2110(__len__)
      3      0.000      0.000      0.503      0.168  peewee.py:4706(__len__)
      600      0.002      0.000      0.471      0.001  peewee.py:7423(get)
      8800      0.038      0.000      0.429      0.000  peewee.py:8012(process_row)
106818/105618      0.035      0.000      0.428      0.000  peewee.py:1444(__sql__)
      106812      0.194      0.000      0.392      0.000  peewee.py:644(value)
      109600      0.026      0.000      0.390      0.000  peewee.py:4966(python_value)
      806      0.009      0.000      0.385      0.000  peewee.py:2500(__sql__)
      27200      0.009      0.000      0.349      0.000  peewee.py:5445(adapt)
      9600      0.020      0.000      0.338      0.000  peewee.py:5410(format_date_time)
      606      0.002      0.000      0.333      0.001  peewee.py:2252(_execute)
      28800      0.011      0.000      0.316      0.000  {built-in method strptime}
      28800      0.026      0.000      0.305      0.000  _strptime.py:564(
_strptime_datetime)
      28800      0.160      0.000      0.279      0.000  _strptime.py:309(_strptime)

cleanup_targets=0 changed_files=200 milvus_inserted=8000 target=
publish_incremental

==> HybridSearchStrategy targets
--- target=search size=medium repeat=2 ---
131064 function calls (124260 primitive calls) in 0.062 seconds

Ordered by: cumulative time
List reduced from 371 to 40 due to restriction <40>

ncalls  tottime  percall  cumtime  percall  filename:lineno(function)
      1      0.000      0.000      0.062      0.062  runners.py:160(run)
      3      0.000      0.000      0.062      0.021  base_events.py:618(
run_until_complete)
      1      0.000      0.000      0.062      0.062  runners.py:86(run)
      3      0.000      0.000      0.062      0.021  base_events.py:594(run_forever)
      6      0.000      0.000      0.062      0.010  base_events.py:1859(_run_once)
      6      0.000      0.000      0.062      0.010  events.py:82(_run)
      6      0.000      0.000      0.062      0.010  {method 'run' of '_contextvars.
Context' objects}
      1      0.000      0.000      0.061      0.061  profiling.py:89(_runner)
      2      0.000      0.000      0.061      0.031  hybrid_strategy.py:34(search)
      2      0.000      0.000      0.042      0.021  hybrid_strategy.py:165(
_filter_vector_results)
      2      0.000      0.000      0.042      0.021  hybrid_strategy.py:196(
_get_active_publish_map)
      6      0.000      0.000      0.041      0.007  peewee.py:2004(inner)
      6      0.000      0.000      0.041      0.007  peewee.py:2079(execute)
      6      0.000      0.000      0.041      0.007  peewee.py:2252(_execute)
      6      0.000      0.000      0.040      0.007  peewee.py:3309(execute)
      4      0.000      0.000      0.039      0.010  peewee.py:7415(__iter__)

```

```

4732/6      0.003      0.000      0.037      0.006 peewee.py:632(sql)
6           0.000      0.000      0.037      0.006 peewee.py:2500(__sql__)
962/6       0.005      0.000      0.035      0.006 peewee.py:1524(__sql__)
534         0.000      0.000      0.016      0.000 peewee.py:4975(__sql__)
534         0.002      0.000      0.015      0.000 peewee.py:1312(__sql__)
208         0.000      0.000      0.008      0.000 peewee.py:4799(next)
208         0.000      0.000      0.008      0.000 peewee.py:4713(iterate)
1530        0.002      0.000      0.007      0.000 peewee.py:602(__call__)
202         0.000      0.000      0.007      0.000 peewee.py:8054(process_row)
540         0.001      0.000      0.005      0.000 peewee.py:1020(__sql__)
2           0.000      0.000      0.005      0.002 hybrid_strategy.py:86(<dictcomp>)
202         0.001      0.000      0.005      0.000 peewee.py:8012(process_row)
1530        0.003      0.000      0.004      0.000 peewee.py:548(__call__)
570/490     0.000      0.000      0.004      0.000 peewee.py:1444(__sql__)
534         0.001      0.000      0.004      0.000 peewee.py:840(apply_column)
1104        0.000      0.000      0.004      0.000 {built-in method builtins.next}
2           0.000      0.000      0.004      0.002 hybrid_strategy.py:54(<listcomp>)
566         0.002      0.000      0.004      0.000 peewee.py:644(value)
1138        0.000      0.000      0.004      0.000 peewee.py:4966(python_value)
160         0.000      0.000      0.004      0.000 dataclasses.py:1265(asdict)
1120/160    0.002      0.000      0.004      0.000 dataclasses.py:1289(_asdict_inner)
)
1104        0.001      0.000      0.004      0.000 peewee.py:567(inner)
2           0.001      0.001      0.004      0.002 hybrid_strategy.py:226(
_rrf_fusion)
82          0.000      0.000      0.003      0.000 peewee.py:5445(adapt)

search_snippets=20 total_results=1532
--- target=rrf size=medium repeat=2 ---
10010 function calls in 0.005 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall  filename:lineno(function)
2       0.002    0.001    0.005    0.003  hybrid_strategy.py:226(
_rrf_fusion)
2       0.002    0.001    0.002    0.001  hybrid_strategy.py:250(<listcomp
>)
2       0.001    0.000    0.001    0.000  {built-in method builtins.sorted}
5000    0.000    0.000    0.000    0.000  hybrid_strategy.py:248(<lambda>)
5000    0.000    0.000    0.000    0.000  {method 'get' of 'dict' objects}
1       0.000    0.000    0.000    0.000  {method 'disable' of '_lsprof.
Profiler' objects}
2       0.000    0.000    0.000    0.000  {method 'keys' of 'dict' objects}
1       0.000    0.000    0.000    0.000  {built-in method builtins.max}

rrf_results=2500 vector=1500 keyword=1000
--- target=filter size=medium repeat=2 ---
14008 function calls in 0.006 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall  filename:lineno(function)
2       0.003    0.002    0.006    0.003  hybrid_strategy.py:165(
_filter_vector_results)
2       0.002    0.001    0.002    0.001  hybrid_strategy.py:170(<listcomp
>)
12000   0.001    0.000    0.001    0.000  typing.py:2287(cast)
2000    0.000    0.000    0.000    0.000  {method 'append' of 'list'
objects}
1       0.000    0.000    0.000    0.000  {built-in method builtins.max}
1       0.000    0.000    0.000    0.000  {method 'disable' of '_lsprof.
Profiler' objects}
2       0.000    0.000    0.000    0.000  profile_hybrid_strategy.py:231(<
lambda>)

filtered_results=1000 source_results=3000

```

B Profiling Benchmarks After Improvements

```
==> SlidingWindowChunker
    2822 function calls in 0.006 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall  filename:lineno(function)
      3    0.001    0.000    0.006    0.002  sliding_window_chunker.py:20(chunk_file)
      3    0.002    0.001    0.002    0.001  {method 'splitlines' of 'str' objects}
    468    0.001    0.000    0.001    0.000  {method 'join' of 'str' objects}
      3    0.001    0.000    0.001    0.000  {method 'strip' of 'str' objects}
    468    0.000    0.000    0.000    0.000  utils.py:6(compute_chunk_hash)
    468    0.000    0.000    0.000    0.000  {built-in method xxhash._xxhash.xxh64_hexdigest}
    468    0.000    0.000    0.000    0.000  <string>:2(__init__)
    468    0.000    0.000    0.000    0.000  {built-in method builtins.min}
    468    0.000    0.000    0.000    0.000  {method 'append' of 'list' objects}
      3    0.000    0.000    0.000    0.000  {built-in method builtins.len}
      1    0.000    0.000    0.000    0.000  {built-in method builtins.max}
      1    0.000    0.000    0.000    0.000  {method 'disable' of '_lsprof.Profiler' objects}

generated_chunks=156 repeat=3 line_count=25000

==> GitOperations.list_files
    327670 function calls (327430 primitive calls) in 0.158 seconds

Ordered by: cumulative time
List reduced from 46 to 40 due to restriction <40>

ncalls  tottime  percall  cumtime  percall  filename:lineno(function)
      2    0.017    0.009    0.158    0.079  git_ops.py:71(list_files)
    9720    0.003    0.000    0.054    0.000  pathlib.py:1008(stat)
    9720    0.003    0.000    0.051    0.000  pathlib.py:765(__truediv__)
    9720    0.027    0.000    0.051    0.000  {built-in method posix.stat}
    9720    0.010    0.000    0.049    0.000  pathlib.py:530(_make_child)
    9842    0.011    0.000    0.034    0.000  pathlib.py:484(_parse_args)
    9722    0.003    0.000    0.024    0.000  pathlib.py:546(_fspath__)
    9722    0.014    0.000    0.021    0.000  pathlib.py:536(_str__)
    9842    0.014    0.000    0.020    0.000  pathlib.py:56(parse_parts)
    9840    0.007    0.000    0.018    0.000  <frozen posixpath>:117(splittext)
    364/124 0.005    0.000    0.014    0.000  <frozen os>:345(_walk)
    9840    0.007    0.000    0.010    0.000  <frozen genericpath>:121(_splittext)
    9720    0.005    0.000    0.007    0.000  pathlib.py:523(_format_parsed_parts)
    10202    0.005    0.000    0.005    0.000  {built-in method builtins.next}
    9720    0.003    0.000    0.004    0.000  pathlib.py:515(_from_parsed_parts)
    10206    0.003    0.000    0.003    0.000  {built-in method sys.intern}
   39970    0.003    0.000    0.003    0.000  {method 'append' of 'list' objects}
   29644    0.002    0.000    0.002    0.000  {built-in method builtins.isinstance}
   19680    0.002    0.000    0.002    0.000  {method 'rfind' of 'str' objects}
    9842    0.002    0.000    0.002    0.000  pathlib.py:239(splitroot)
    9720    0.002    0.000    0.002    0.000  pathlib.py:94(join_parsed_parts)
     122    0.000    0.000    0.002    0.000  pathlib.py:868(_new__)
   19680    0.001    0.000    0.001    0.000  {method 'lower' of 'str' objects}
    9720    0.001    0.000    0.001    0.000  {method 'join' of 'str' objects}
     122    0.000    0.000    0.001    0.000  pathlib.py:504(_from_parts)
   19804    0.001    0.000    0.001    0.000  {built-in method posix.fspath}
    9842    0.001    0.000    0.001    0.000  {built-in method _new__ of type object at 0x559079f36f10}
     122    0.001    0.000    0.001    0.000  {built-in method posix.scandir}
    9842    0.001    0.000    0.001    0.000  {method 'reverse' of 'list' objects}
     120    0.000    0.000    0.001    0.000  <frozen posixpath>:164(islink)
```

```

10080      0.001      0.000      0.001      0.000 {method 'is_dir' of 'posix.
DirEntry' objects}
120      0.000      0.000      0.000      0.000 <frozen posixpath>:71(join)
120      0.000      0.000      0.000      0.000 {built-in method posix.lstat}
122      0.000      0.000      0.000      0.000 {method 'split' of 'str' objects}
120      0.000      0.000      0.000      0.000 <frozen posixpath>:41(_get_sep)
122      0.000      0.000      0.000      0.000 git_ops.py:81(<listcomp>)
120      0.000      0.000      0.000      0.000 {method 'endswith' of 'str'
objects}
120      0.000      0.000      0.000      0.000 {method 'startswith' of 'str'
objects}
122      0.000      0.000      0.000      0.000 {method 'lstrip' of 'str' objects
}
244      0.000      0.000      0.000      0.000 {built-in method builtins.len}

listed_files=4860 repeat=2 dirs=60 files_per_dir=80

==> ChunkService.move_to_final
      3404358 function calls (3291622 primitive calls) in 1.260 seconds

Ordered by: cumulative time
List reduced from 230 to 40 due to restriction <40>

ncalls      tottime      percall      cumtime      percall      filename:lineno(function)
1          0.010          0.010          1.260          1.260      chunk.py:107(move_to_final)
18          0.000          0.000          1.087          0.060      peewee.py:2004(inner)
18          0.000          0.000          1.087          0.060      peewee.py:2079(execute)
17          0.002          0.000          1.085          0.064      peewee.py:2595(_execute)
18          0.000          0.000          1.084          0.060      peewee.py:3309(execute)
16          0.000          0.000          1.053          0.066      peewee.py:2881(_execute)
104753/18    0.040          0.000          0.961          0.053      peewee.py:632(sql)
16          0.005          0.000          0.960          0.060      peewee.py:2848(__sql__)
16          0.262          0.016          0.954          0.060      peewee.py:2723(_generate_insert)
8035/35     0.056          0.000          0.551          0.016      peewee.py:1868(__sql__)
96002       0.031          0.000          0.380          0.000      peewee.py:1444(__sql__)
96002       0.171          0.000          0.350          0.000      peewee.py:644(value)
18          0.000          0.000          0.121          0.007      peewee.py:3302(execute_sql)
18          0.101          0.006          0.121          0.007      {method 'execute' of 'sqlite3.
Cursor' objects}
24328/24327 0.002          0.000          0.079          0.000      {built-in method builtins.len}
224384      0.060          0.000          0.077          0.000      peewee.py:4939(__hash__)
1          0.000          0.000          0.077          0.077      peewee.py:2110(__len__)
1          0.000          0.000          0.077          0.077      peewee.py:4706(__len__)
1          0.002          0.002          0.077          0.077      peewee.py:4739(fill_cache)
16002       0.005          0.000          0.077          0.000      peewee.py:4799(next)
8000        0.005          0.000          0.076          0.000      chunk.py:38(_decode_embedding)
8001        0.006          0.000          0.071          0.000      peewee.py:4713(iterate)
8000        0.004          0.000          0.069          0.000      __init__.py:299(loads)
8000        0.016          0.000          0.064          0.000      decoder.py:332(decode)
208530      0.042          0.000          0.052          0.000      peewee.py:640(literal)
577613      0.044          0.000          0.052          0.000      {built-in method builtins.
isinstance}
192285      0.038          0.000          0.051          0.000      peewee.py:562(__getattr__)
8000        0.005          0.000          0.049          0.000      peewee.py:8031(process_row)
96002       0.023          0.000          0.046          0.000      peewee.py:4963(db_value)
8000        0.016          0.000          0.044          0.000      peewee.py:8033(<listcomp>)
8000        0.043          0.000          0.043          0.000      decoder.py:343(raw_decode)
96032       0.020          0.000          0.041          0.000      peewee.py:396(<lambda>)
80000       0.015          0.000          0.028          0.000      peewee.py:4966(python_value)
8087        0.009          0.000          0.027          0.000      peewee.py:602(__call__)
120002      0.021          0.000          0.026          0.000      peewee.py:5115(adapt)
432881      0.023          0.000          0.023          0.000      {method 'append' of 'list'
objects}
96032       0.016          0.000          0.022          0.000      inspect.py:292(isclass)
16000       0.002          0.000          0.019          0.000      dbapi2.py:58(adapt_datetime)
16000       0.017          0.000          0.017          0.000      {method 'isoformat' of 'datetime.
datetime' objects}
224385      0.017          0.000          0.017          0.000      {built-in method builtins.hash}

moved=8000 final_count=8000 milvus_inserted=8000 target=move_to_final

==> ChunkService.publish_incremental_batch

```

3602787 function calls (3480566 primitive calls) in 1.436 seconds

Ordered by: cumulative time

List reduced from 261 to 30 due to restriction <30>

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
3	0.011	0.004	1.436	0.479	chunk.py:200(publish_incremental_batch)
28	0.000	0.000	1.128	0.040	peewee.py:2004(inner)
28	0.000	0.000	1.128	0.040	peewee.py:2079(execute)
28	0.000	0.000	1.125	0.040	peewee.py:3309(execute)
22	0.002	0.000	1.113	0.051	peewee.py:2595(_execute)
19	0.000	0.000	1.091	0.057	peewee.py:2881(_execute)
111224/28	0.046	0.000	1.000	0.036	peewee.py:632(sql)
19	0.006	0.000	0.987	0.052	peewee.py:2848(__sql__)
19	0.181	0.010	0.978	0.051	peewee.py:2723(_generate_insert)
8674/62	0.066	0.000	0.645	0.010	peewee.py:1868(__sql__)
101418/100218	0.040	0.000	0.446	0.000	peewee.py:1444(__sql__)
101412	0.198	0.000	0.405	0.000	peewee.py:644(value)
8000	0.005	0.000	0.196	0.000	chunk.py:40(_decode_embedding)
8000	0.005	0.000	0.190	0.000	__init__.py:299(loads)
8000	0.008	0.000	0.184	0.000	decoder.py:332(decode)
8000	0.171	0.000	0.171	0.000	decoder.py:343(raw_decode)
34	0.000	0.000	0.125	0.004	peewee.py:3302(execute_sql)
34	0.100	0.003	0.124	0.004	{method 'execute' of 'sqlite3. Cursor' objects}
26347/26346	0.002	0.000	0.090	0.000	{built-in method builtins.len}
16409	0.006	0.000	0.089	0.000	peewee.py:4799(next)
235232	0.069	0.000	0.089	0.000	peewee.py:4939(__hash__)
3	0.000	0.000	0.088	0.029	peewee.py:2110(__len__)
3	0.000	0.000	0.088	0.029	peewee.py:4706(__len__)
3	0.002	0.001	0.088	0.029	peewee.py:4739(fill_cache)
8406	0.007	0.000	0.083	0.000	peewee.py:4713(iterate)
220920	0.048	0.000	0.060	0.000	peewee.py:640(literal)
205728	0.045	0.000	0.060	0.000	peewee.py:562(_getattr__)
8400	0.006	0.000	0.057	0.000	peewee.py:8031(process_row)
610410	0.047	0.000	0.055	0.000	{built-in method builtins. isinstance}
101412	0.026	0.000	0.054	0.000	peewee.py:4963(db_value)

cleanup_targets=0 changed_files=200 milvus_inserted=8000 target=
publish_incremental

==> HybridSearchStrategy targets

--- target=search size=medium repeat=2 keyword_mode=stubbed ---
54717 function calls (52159 primitive calls) in 0.023 seconds

Ordered by: cumulative time

List reduced from 373 to 40 due to restriction <40>

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
1	0.000	0.000	0.023	0.023	runners.py:160(run)
3	0.000	0.000	0.023	0.008	base_events.py:618(run_until_complete)
1	0.000	0.000	0.023	0.023	runners.py:86(run)
3	0.000	0.000	0.023	0.008	base_events.py:594(run_forever)
6	0.000	0.000	0.023	0.004	base_events.py:1859(_run_once)
6	0.000	0.000	0.022	0.004	events.py:82(_run)
6	0.000	0.000	0.022	0.004	{method 'run' of '_contextvars. Context' objects}
1	0.000	0.000	0.022	0.022	profiling.py:89(_runner)
2	0.000	0.000	0.022	0.011	hybrid_strategy.py:34(search)
6	0.000	0.000	0.007	0.001	peewee.py:2004(inner)
6	0.000	0.000	0.007	0.001	peewee.py:2079(execute)
6	0.000	0.000	0.007	0.001	peewee.py:2252(_execute)
6	0.000	0.000	0.007	0.001	peewee.py:3309(execute)
794/6	0.000	0.000	0.006	0.001	peewee.py:632(sql)
4	0.000	0.000	0.006	0.001	peewee.py:7415(__iter__)
6	0.000	0.000	0.006	0.001	peewee.py:2500(__sql__)
2	0.000	0.000	0.006	0.003	hybrid_strategy.py:171(_filter_vector_results)
208	0.000	0.000	0.006	0.000	peewee.py:4799(next)

208	0.000	0.000	0.006	0.000	peewee.py:4713(iterate)
2	0.000	0.000	0.005	0.003	hybrid_strategy.py:203(
_get_active_publish_map)					
2	0.000	0.000	0.004	0.002	hybrid_strategy.py:89(<dictcomp>)
20	0.000	0.000	0.004	0.000	peewee.py:1868(__sql__)
42	0.000	0.000	0.004	0.000	peewee.py:8054(process_row)
2	0.000	0.000	0.004	0.002	hybrid_strategy.py:54(<listcomp>)
160	0.000	0.000	0.004	0.000	dataclasses.py:1265(asdict)
1120/160	0.001	0.000	0.004	0.000	dataclasses.py:1289(_asdict_inner
)					
14/6	0.000	0.000	0.004	0.001	peewee.py:1524(__sql__)
42	0.000	0.000	0.004	0.000	peewee.py:8012(process_row)
1138	0.000	0.000	0.004	0.000	peewee.py:4966(python_value)
82	0.000	0.000	0.003	0.000	peewee.py:5445(adapt)
82	0.000	0.000	0.003	0.000	peewee.py:5410(format_date_time)
2	0.001	0.001	0.003	0.001	hybrid_strategy.py:244(
_rrf_fusion)					
246	0.000	0.000	0.003	0.000	{built-in method strptime}
246	0.000	0.000	0.003	0.000	_strptime.py:564(
_strptime_datetime)					
418/18	0.000	0.000	0.003	0.000	peewee.py:1444(__sql__)
246	0.001	0.000	0.002	0.000	_strptime.py:309(_strptime)
410	0.001	0.000	0.002	0.000	peewee.py:644(value)
2	0.000	0.000	0.002	0.001	peewee.py:6967(get_or_none)
2	0.000	0.000	0.002	0.001	peewee.py:6954(get)
2	0.000	0.000	0.002	0.001	peewee.py:7423(get)

search_snippets=20 total_results=1532

--- target=rrf size=medium repeat=2 keyword_mode=stubbed ---
10010 function calls in 0.004 seconds

Ordered by: cumulative time

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
2	0.002	0.001	0.004	0.002	hybrid_strategy.py:244(
_rrf_fusion)					
2	0.001	0.001	0.001	0.001	hybrid_strategy.py:268(<listcomp
>)					
2	0.000	0.000	0.001	0.000	{built-in method builtins.sorted}
5000	0.000	0.000	0.000	0.000	hybrid_strategy.py:266(<lambda>)
5000	0.000	0.000	0.000	0.000	{method 'get' of 'dict' objects}
1	0.000	0.000	0.000	0.000	{built-in method builtins.max}
2	0.000	0.000	0.000	0.000	{method 'keys' of 'dict' objects}
1	0.000	0.000	0.000	0.000	{method 'disable' of '_lsprof.
Profiler' objects}					

rrf_results=2500 vector=1500 keyword=1000

--- target=filter size=medium repeat=2 keyword_mode=stubbed ---
14008 function calls in 0.006 seconds

Ordered by: cumulative time

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
2	0.003	0.002	0.006	0.003	hybrid_strategy.py:171(
_filter_vector_results)					
2	0.002	0.001	0.002	0.001	hybrid_strategy.py:176(<setcomp>)
12000	0.000	0.000	0.000	0.000	typing.py:2287(cast)
2000	0.000	0.000	0.000	0.000	{method 'append' of 'list'
objects}					
1	0.000	0.000	0.000	0.000	{built-in method builtins.max}
1	0.000	0.000	0.000	0.000	{method 'disable' of '_lsprof.
Profiler' objects}					
2	0.000	0.000	0.000	0.000	profile_hybrid_strategy.py:253(<
lambda>)					

filtered_results=1000 source_results=3000